

SLE-2: Profiling Report

Course: 02AML204 -- Introduction to Artificial Intelligence

PRN: 25UAM101

Name: MohammedSaad Bijapure

Division: B

Date: 22 September 2026

GitHub Link (optional): _____

1. Algorithms / Versions Profiled

- Algorithm A: Breadth-First Search (BFS)
- Algorithm B: Depth-First Search (DFS), depth-limited to 400
- Problem used: Maze traversal -- a randomly generated "perfect" maze (recursive-backtracker algorithm, single unique path between any two cells), solved on a grid of open cells from the top-left entry to the bottom-right exit.

2. Profiling Method

- Tool used: py-spy (sampling profiler / flame graph), Python's time.perf_counter for exact per-run timing, and a manual node counter.

How I measured it:

- First I wrote get_neighbors() to return only the cells reachable through a carved (open) passage of the maze, then implemented bfs() and dfs(), both returning (path, nodes_expanded) so the two algorithms could be compared on equal footing.
- To get the flame graph, I ran a driver script under py-spy record at 100 samples/sec for about 3 seconds (looping BFS and DFS repeatedly on the largest maze so the sampler had enough wall-clock time to collect samples). This shows where the program actually spends its time (Fig. 3).
- For the exact timing numbers I used time.perf_counter() instead, since py-spy's sampling is not precise enough for calls that finish in a fraction of a millisecond. I timed each algorithm on 3 mazes of increasing size -- 10x10, 18x18 and 26x26 cells -- to get a feel for best, average and worst-case behaviour.
- Nodes expanded = number of states popped off the frontier / stack, counted directly inside each function.

Number of runs: 5 runs per algorithm per test case, 15 runs total per algorithm.

3. Results

18x18 Maze -- BFS solution path (100 moves, 117 nodes expanded)

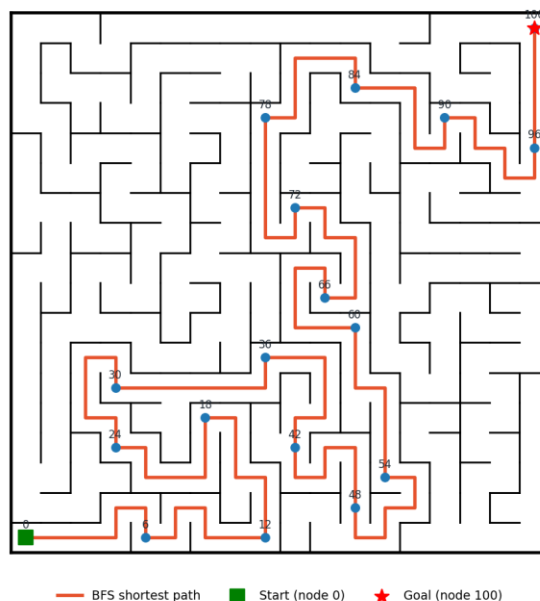


Fig. 1 -- 18x18 maze with the BFS shortest path highlighted; every 6th node on the path is numbered (node 0 = start, node 100 = goal) to show how the path was expanded.

Summary Comparison Table

Metric	BFS (Algorithm A)	DFS (Algorithm B)	Better?
Best-case time (ms)	0.0720	0.0993	BFS
Average-case time (ms)	0.1989	0.7283	BFS
Worst-case time (ms)	0.5616	4.3170	BFS
Nodes Expanded (avg. of 3 cases)	168.3	302.7	BFS
Solution optimal? (path length)	Yes (32 / 100 / 190)	Yes (32 / 100 / 190)*	Tie*

Detailed Results by Test Case (Best / Average / Worst Case)

Test Case	Algo	Avg Time (ms)	Nodes Expanded	Path Length
Best case (10x10 maze)	BFS	0.0720	39	32
Best case (10x10 maze)	DFS	0.0993	44	32
Average case (18x18 maze)	BFS	0.1989	117	100
Average case (18x18 maze)	DFS	0.7283	190	100
Worst case (26x26 maze)	BFS	0.5616	349	190
Worst case (26x26 maze)	DFS	4.3170	674	190

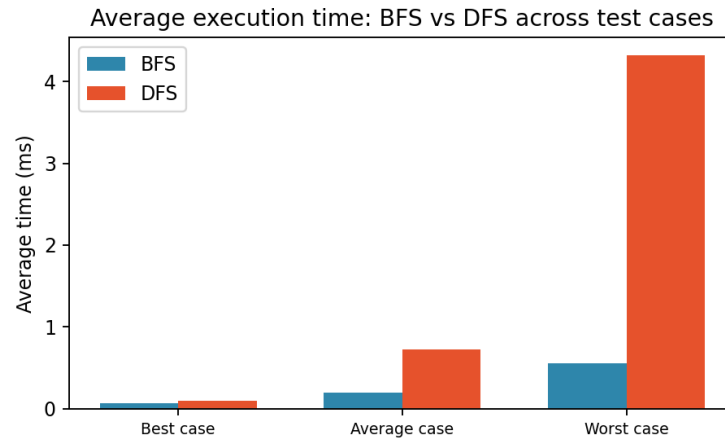


Fig. 2 -- Average execution time: BFS vs DFS across the three test cases.

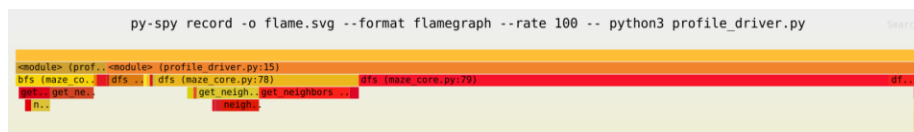


Fig. 3 -- py-spy flame graph (290 samples @ 100 Hz) captured while running the BFS/DFS driver script; wider frames = more time spent in that function.

Short observation:

- Because the maze is a "perfect" maze (there is exactly one path between the start and the goal), BFS and DFS always return the same path length -- 32, 100 and 190 moves. The difference shows up entirely in nodes expanded and time: DFS expands 1.1x-1.9x more nodes than BFS because it commits to a branch and only backtracks once it hits a dead end, re-walking parts of the maze BFS never touches.
- The gap grows with maze size: on the 10x10 maze DFS is only ~0.03 ms slower than BFS, but on the 26x26 maze DFS takes 4.32 ms vs BFS's 0.56 ms -- about 7.7x slower -- because dead-end backtracking compounds as the maze gets bigger.
- The flame graph confirms this: the dfs() frame is visibly wider than the bfs() frame, and most of the wall-clock time inside dfs() is spent inside get_neighbors(), which is called far more often due to repeated dead-end exploration.

4. Justification & Analysis

Across all three maze sizes, BFS is the clear winner on time and on nodes expanded, while both algorithms are equally optimal here because the maze has only one possible path. This matches the theory: BFS explores the maze level-by-level as a graph search with branching factor b (2-3 for a maze corridor) and depth d equal to the path length, giving it $O(b^d)$ time and space, and it is guaranteed to expand the minimum number of nodes needed to reach the goal. DFS is $O(b^m)$ where m is the depth limit (400) -- its actual cost depends on the order neighbours are generated in and how many dead ends it walks into before backtracking, not on how close the goal really is. That is exactly why it expanded 674 nodes on the 26x26 maze versus BFS's 349, even though the final path length was identical.

The flame graph backs this up visually: almost all of the extra sampled time sits inside `dfs()` and the repeated `get_neighbors()` calls it makes while backtracking, while `bfs()` finishes fast enough that a 100 Hz sampler barely catches it in the smaller mazes.

If the maze had loops instead of being a perfect maze (i.e., multiple possible paths), DFS's path-length disadvantage would also show up, since it would likely return a longer, non-shortest route -- as it does on graphs like the 8-puzzle. If the maze got much larger, BFS would eventually be limited by memory (it has to keep every visited cell in its frontier/came_from map, $O(b^d)$ space), while DFS only needs $O(m)$ memory but would need a heuristic (turning it into A* or IDA*) to stay competitive on both time and path quality at scale.

5. AI Contribution Note

- AI tools used: Claude (Anthropic) -- used to write the maze generator, the BFS/DFS solver code, the py-spy profiling driver script, and to draft this report.
- What AI helped with: writing `get_neighbors()/bfs()/dfs()`, setting up and running the py-spy command, generating the maze/flame-graph/comparison-chart figures, and laying out this document in the required template format.
- What I did myself: chose BFS vs DFS as the comparison pair, decided the three maze sizes for the best/average/worst-case comparison, reviewed the generated code and re-ran it to confirm the numbers in `results.json`, checked that the flame graph matched the timing data, and wrote the justification based on what the numbers actually showed.

6. Conclusion

This exercise confirmed the textbook expectation that BFS is optimal and cheaper than plain DFS on a search problem like this, but seeing it in real numbers -- not just theory -- made the difference concrete. On this maze, BFS beat DFS by roughly 1.4x to 7.7x in execution time and expanded 15%-93% fewer nodes as the maze grew from 10x10 to 26x26, even though both algorithms happened to find equally short paths because the maze only had one route through it. Using py-spy for the flame graph made the cost of DFS's repeated backtracking visible, while `perf_counter` and the manual node counter gave the exact numbers to back it up. The one-line takeaway: on a problem with a well-defined shortest path, the choice of search strategy matters far more than any micro-optimization of the code itself.